

My Project

Generated by Doxygen 1.13.2

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 Vector< T, Allocator > Class Template Reference	5
3.1.1 Member Typedef Documentation	6
3.1.1.1 allocator_type	6
3.1.1.2 const_iterator	6
3.1.1.3 const_pointer	6
3.1.1.4 const_reference	6
3.1.1.5 difference_type	7
3.1.1.6 iterator	7
3.1.1.7 pointer	7
3.1.1.8 reference	7
3.1.1.9 reverse_iterator	7
3.1.1.10 size_type	7
3.1.1.11 value_type	7
3.1.2 Constructor & Destructor Documentation	7
3.1.2.1 Vector() [1/5]	7
3.1.2.2 Vector() [2/5]	8
3.1.2.3 Vector() [3/5]	8
3.1.2.4 Vector() [4/5]	8
3.1.2.5 Vector() [5/5]	8
3.1.2.6 ~Vector()	8
3.1.3 Member Function Documentation	8
3.1.3.1 append_range()	8
3.1.3.2 assign()	8
3.1.3.3 at() [1/2]	9
3.1.3.4 at() [2/2]	9
3.1.3.5 back() [1/2]	9
3.1.3.6 back() [2/2]	9
3.1.3.7 begin() [1/2]	9
3.1.3.8 begin() [2/2]	9
3.1.3.9 capacity()	9
3.1.3.10 clear()	9
3.1.3.11 data() [1/2]	10
3.1.3.12 data() [2/2]	10
3.1.3.13 empty()	10
3.1.3.14 end() [1/2]	10
3.1.3.15 end() [2/2]	10

3.1.3.16 erase() [1/2]	10
3.1.3.17 erase() [2/2]	10
3.1.3.18 front() [1/2]	10
3.1.3.19 front() [2/2]	11
3.1.3.20 insert()	11
3.1.3.21 max_size()	11
3.1.3.22 operator=() [1/2]	11
3.1.3.23 operator=() [2/2]	11
3.1.3.24 operator[]()	11
3.1.3.25 pop_back()	11
3.1.3.26 push_back()	11
3.1.3.27 reserve()	12
3.1.3.28 resize() [1/2]	12
3.1.3.29 resize() [2/2]	12
3.1.3.30 shrink_to_fit()	12
3.1.3.31 size()	12
3.1.3.32 swap()	12
4 File Documentation	13
4.1 vector.h File Reference	13
4.1.1 Function Documentation	13
4.1.1.1 operator!=(())	13
4.1.1.2 operator==(())	13
4.2 vector.h	14
Index	19

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Vector< T, Allocator >	5
--	---

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

vector.h	13
------------------------------------	----

Chapter 3

Class Documentation

3.1 `Vector< T, Allocator >` Class Template Reference

```
#include <vector.h>
```

Public Types

- using `value_type` = `T`
- using `pointer` = `T*`
- using `iterator` = `pointer`
- using `allocator_type` = `Allocator`
- using `size_type` = `std::size_t`
- using `difference_type` = `std::ptrdiff_t`
- using `reference` = `value_type&`
- using `const_reference` = `const value_type&`
- using `const_pointer` = `typename Allocator::const_pointer`
- using `const_iterator` = `const_pointer`
- using `reverse_iterator` = `std::reverse_iterator<iterator>`

Public Member Functions

- `Vector` ()
- `Vector` (`T` array[], `int` n)
- `Vector` (`std::initializer_list< T >` init)
- `Vector` (`const Vector` &other)
- `Vector` & `operator=` (`const Vector` &other)
- `Vector` (`Vector` &&other) `noexcept`
- `Vector` & `operator=` (`Vector` &&other) `noexcept`
- `~Vector` ()
- `void assign` (`size_type` count, `const T` &value)
- `T` & `at` (`int` pos)
- `const T` & `at` (`int` pos) `const`
- `T` & `operator[]` (`size_t` index)
- `T` & `front` ()
- `T` & `back` ()
- `const T` & `front` () `const`

- `const T & back () const`
- `T * data ()`
- `const T * data () const`
- `T * begin ()`
- `const T * begin () const`
- `T * end ()`
- `const T * end () const`
- `bool empty ()`
- `size_type size () const`
- `size_type max_size () const`
- `void reserve (size_type new_cap)`
- `size_type capacity () const`
- `void shrink_to_fit ()`
- `void clear ()`
- `iterator insert (iterator pos, const T &value)`
- `iterator erase (const_iterator pos)`
- `iterator erase (const_iterator first, const_iterator last)`
- `void push_back (const T &item)`
- `void append_range ()`
- `void pop_back ()`
- `void resize ()`
- `void resize (size_t capacity)`
- `void swap (Vector &other)`

3.1.1 Member Typedef Documentation

3.1.1.1 allocator_type

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::allocator_type = Allocator
```

3.1.1.2 const_iterator

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_iterator = const_pointer
```

3.1.1.3 const_pointer

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_pointer = typename Allocator::const_pointer
```

3.1.1.4 const_reference

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_reference = const value_type&
```

3.1.1.5 difference_type

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::difference_type = std::ptrdiff_t
```

3.1.1.6 iterator

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::iterator = pointer
```

3.1.1.7 pointer

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::pointer = T*
```

3.1.1.8 reference

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::reference = value_type&
```

3.1.1.9 reverse_iterator

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::reverse_iterator = std::reverse_iterator<iterator>
```

3.1.1.10 size_type

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::size_type = std::size_t
```

3.1.1.11 value_type

```
template<typename T, typename Allocator = std::allocator<T>>
using Vector< T, Allocator >::value_type = T
```

3.1.2 Constructor & Destructor Documentation

3.1.2.1 Vector() [1/5]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector () [inline]
```

3.1.2.2 Vector() [2/5]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    T array[],
    int n) [inline]
```

3.1.2.3 Vector() [3/5]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    std::initializer_list< T > init) [inline]
```

3.1.2.4 Vector() [4/5]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    const Vector< T, Allocator > & other) [inline]
```

3.1.2.5 Vector() [5/5]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    Vector< T, Allocator > && other) [inline], [noexcept]
```

3.1.2.6 ~Vector()

```
template<typename T, typename Allocator = std::allocator<T>>
Vector< T, Allocator >::~~Vector () [inline]
```

3.1.3 Member Function Documentation

3.1.3.1 append_range()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::append_range () [inline]
```

3.1.3.2 assign()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::assign (
    size_type count,
    const T & value) [inline]
```

3.1.3.3 at() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
T & Vector< T, Allocator >::at (
    int pos) [inline]
```

3.1.3.4 at() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const T & Vector< T, Allocator >::at (
    int pos) const [inline]
```

3.1.3.5 back() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
T & Vector< T, Allocator >::back () [inline]
```

3.1.3.6 back() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const T & Vector< T, Allocator >::back () const [inline]
```

3.1.3.7 begin() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
T * Vector< T, Allocator >::begin () [inline]
```

3.1.3.8 begin() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const T * Vector< T, Allocator >::begin () const [inline]
```

3.1.3.9 capacity()

```
template<typename T, typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::capacity () const [inline]
```

3.1.3.10 clear()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::clear () [inline]
```

3.1.3.11 data() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
T * Vector< T, Allocator >::data () [inline]
```

3.1.3.12 data() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const T * Vector< T, Allocator >::data () const [inline]
```

3.1.3.13 empty()

```
template<typename T, typename Allocator = std::allocator<T>>
bool Vector< T, Allocator >::empty () [inline]
```

3.1.3.14 end() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
T * Vector< T, Allocator >::end () [inline]
```

3.1.3.15 end() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const T * Vector< T, Allocator >::end () const [inline]
```

3.1.3.16 erase() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::erase (
    const_iterator first,
    const_iterator last) [inline]
```

3.1.3.17 erase() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::erase (
    const_iterator pos) [inline]
```

3.1.3.18 front() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
T & Vector< T, Allocator >::front () [inline]
```

3.1.3.19 front() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
const T & Vector< T, Allocator >::front () const [inline]
```

3.1.3.20 insert()

```
template<typename T, typename Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    iterator pos,
    const T & value) [inline]
```

3.1.3.21 max_size()

```
template<typename T, typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::max_size () const [inline]
```

3.1.3.22 operator=() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    const Vector< T, Allocator > & other) [inline]
```

3.1.3.23 operator=() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    Vector< T, Allocator > && other) [inline], [noexcept]
```

3.1.3.24 operator[]()

```
template<typename T, typename Allocator = std::allocator<T>>
T & Vector< T, Allocator >::operator[] (
    size_t index) [inline]
```

3.1.3.25 pop_back()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::pop_back () [inline]
```

3.1.3.26 push_back()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::push_back (
    const T & item) [inline]
```

3.1.3.27 reserve()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::reserve (
    size_type new_cap) [inline]
```

3.1.3.28 resize() [1/2]

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::resize () [inline]
```

3.1.3.29 resize() [2/2]

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::resize (
    size_t capacity) [inline]
```

3.1.3.30 shrink_to_fit()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::shrink_to_fit () [inline]
```

3.1.3.31 size()

```
template<typename T, typename Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::size () const [inline]
```

3.1.3.32 swap()

```
template<typename T, typename Allocator = std::allocator<T>>
void Vector< T, Allocator >::swap (
    Vector< T, Allocator > & other) [inline]
```

The documentation for this class was generated from the following file:

- [vector.h](#)

Chapter 4

File Documentation

4.1 vector.h File Reference

```
#include <iostream>
```

Classes

- class `Vector< T, Allocator >`

Functions

- `template<typename T>`
`bool operator== (const Vector< T > &lhs, const Vector< T > &rhs)`
- `template<typename T>`
`bool operator!= (const Vector< T > &lhs, const Vector< T > &rhs)`

4.1.1 Function Documentation

4.1.1.1 `operator"!=()`

```
template<typename T>
bool operator!= (
    const Vector< T > & lhs,
    const Vector< T > & rhs)
```

4.1.1.2 `operator==()`

```
template<typename T>
bool operator== (
    const Vector< T > & lhs,
    const Vector< T > & rhs)
```

4.2 vector.h

[Go to the documentation of this file.](#)

```

00001 #include <iostream>
00002
00003 template <typename T, typename Allocator = std::allocator<T>
00004 class Vector{
00005 private:
00006     size_t size_;
00007     size_t capacity_;
00008     T* data_;
00009
00010 public:
00011     using value_type = T;
00012     using pointer = T*;
00013     using iterator = pointer;
00014     using allocator_type = Allocator;
00015     using size_type = std::size_t;
00016     using difference_type = std::ptrdiff_t;
00017     using reference = value_type&;
00018     using const_reference = const value_type&;
00019     using const_pointer = typename Allocator::const_pointer;
00020     using const_iterator = const_pointer;
00021     using reverse_iterator = std::reverse_iterator<iterator>;
00022
00023 //member functions
00024 //constructors
00025     //default constructor
00026     Vector() : size_(0), capacity_(1) { data_ = new T[capacity_];}
00027
00028     //array constructor
00029     Vector(T array[], int n){
00030         data_ = new T[n]; //reserves the same space as the array passed
00031         for(int i = 0; i < n; i++){
00032             data_[i] = array[i];
00033         }
00034         size_ = n;
00035     }
00036
00037     //list constructor
00038     Vector(std::initializer_list<T> init) {
00039         size_ = init.size();
00040         capacity_ = size_;
00041         data_ = new T[capacity_];
00042
00043         size_t i = 0;
00044         for (const T& value : init) {
00045             data_[i] = value;
00046             i++;
00047         }
00048     }
00049
00050     //copy constructor
00051     Vector(const Vector& other): size_(other.size_), capacity_(other.capacity_){
00052         data_ = new T[capacity_];
00053         for(size_t i=0; i<size_; i++) {
00054             data_[i] = other.data_[i];
00055         }
00056     }
00057
00058     //copy assignment operator
00059     Vector& operator=(const Vector& other){
00060         if(this == &other) return *this;
00061
00062         size_ = other.size_;
00063         capacity_ = other.capacity_;
00064         data_ = new T[capacity_];
00065         for(size_t i = 0; i < size_; ++i) {
00066             data_[i] = other.data_[i];
00067         }
00068
00069         return *this;
00070     }
00071
00072     //move constructor
00073     Vector(Vector&& other) noexcept
00074     : data_(other.data_), size_(other.size_), capacity_(other.capacity_) {
00075         other.data_ = nullptr;
00076         other.size_ = 0;
00077         other.capacity_ = 0;
00078     }
00079
00080     //move assignment operator
00081     Vector& operator=(Vector&& other) noexcept{
00082         if(this == &other) return *this;

```

```

00083         delete[] data_;
00084         data_ = other.data_;
00085         size_ = other.size_;
00086         capacity_ = other.capacity_;
00087
00088         other.data_ = nullptr;
00089         other.size_ = 0;
00090         other.capacity_ = 0;
00091
00092         return *this;
00093     }
00094
00095     ~Vector() {
00096         clear();
00097     }
00098
00099     void assign(size_type count, const T& value) {
00100         clear();
00101         if (count > capacity_) {
00102             delete[] data_;
00103             capacity_ = count;
00104             data_ = new T[capacity_];
00105         } else {
00106             if (data_ == nullptr) {
00107                 data_ = new T[capacity_];
00108             }
00109         }
00110
00111         for (size_type i = 0; i < count; ++i) {
00112             data_[i] = value;
00113         }
00114         size_ = count;
00115     }
00116
00117     //element access
00118     T& at(int pos){
00119         if(pos >= size_){
00120             throw std::out_of_range("Index out of bounds");
00121         }
00122         return data_[pos];
00123     }
00124
00125     const T& at(int pos) const {
00126         if (pos >= size_) {
00127             throw std::out_of_range("Index out of bounds");
00128         }
00129         return data_[pos];
00130     }
00131
00132     T& operator[](size_t index){
00133         if (index >= size_){
00134             throw std::out_of_range("Index out of bounds");
00135         }
00136         return data_[index];
00137     }
00138
00139     T& front(){
00140         return at(0);
00141     }
00142
00143     T& back(){
00144         return at(size_-1);
00145     }
00146
00147     const T& front() const{
00148         return at(0);
00149     }
00150
00151     const T& back() const{
00152         return at(size_-1);
00153     }
00154
00155     T* data(){
00156         return data_;
00157     }
00158
00159     const T* data() const {
00160         return data_;
00161     }
00162
00163     //iterators
00164     T* begin(){
00165         return data_;
00166     }
00167
00168     const T* begin() const {
00169         return data_;

```

```

00170     }
00171
00172     T* end(){
00173         return data_ + size_;
00174     }
00175
00176     const T* end() const{
00177         return data_ + size_;
00178     }
00179
00180     //capacity
00181     bool empty(){
00182         return size_ == 0;
00183     }
00184
00185     size_type size() const{ return size_; }
00186
00187     size_type max_size() const{
00188         return std::numeric_limits<size_type>::max() / sizeof(T);
00189     }
00190
00191     void reserve(size_type new_cap ){
00192         if (new_cap > capacity_) {
00193             T* new_data = new T[new_cap];
00194             for (size_type i = 0; i < size_; ++i) {
00195                 new_data[i] = std::move(data_[i]);
00196             }
00197             delete[] data_;
00198             data_ = new_data;
00199             capacity_ = new_cap;
00200         }
00201     }
00202
00203     size_type capacity() const{ return capacity_; }
00204
00205     void shrink_to_fit(){
00206         if (capacity_ > size_) {
00207             T* new_data = new T[size_];
00208             for (size_type i = 0; i < size_; ++i) {
00209                 new_data[i] = std::move(data_[i]);
00210             }
00211             delete[] data_;
00212             data_ = new_data;
00213             capacity_ = size_;
00214         }
00215     }
00216
00217     //modifiers
00218     void clear(){
00219         size_ = 0;
00220         capacity_ = 0;
00221         delete[] data_;
00222         data_ = nullptr;
00223     }
00224
00225     iterator insert(iterator pos, const T& value){
00226         size_t index = pos - begin();
00227
00228         if(index > size_){
00229             throw std::out_of_range("Invalid position");
00230         }
00231         if(size_ == capacity_){
00232             resize();
00233         }
00234         size_++;
00235         for(size_t i = size_-1; i > index; --i){
00236             data_[i]=data_[i-1];
00237         }
00238         data_[index]=value;
00239         ++size_;
00240
00241         return begin() + index;
00242     }
00243
00244     iterator erase(const_iterator pos) {
00245         if(pos < begin() || pos >= end()) {
00246             throw std::out_of_range("Invalid position");
00247         }
00248
00249         size_t index = std::distance(static_cast<const T*>(begin()), pos);
00250
00251         for(size_t i = index; i < size_ - 1; i++) {
00252             data_[i] = data_[i+1];
00253         }
00254         --size_;
00255
00256         return begin() + index;

```

```

00257     }
00258
00259     iterator erase(const_iterator first, const_iterator last ){
00260         size_t pos_first = std::distance(static_cast<const T*>(begin()), first);
00261         size_t pos_last = std::distance(static_cast<const T*>(begin()), last);
00262         size_t count = pos_last - pos_first;
00263
00264         if (count == 0) {
00265             return begin() + pos_first;
00266         }
00267
00268         for (size_t i = pos_first; i + count < size_; ++i) {
00269             data_[i] = data_[i + count];
00270         }
00271
00272         size_ -= count;
00273
00274         return begin() + pos_first;
00275     }
00276
00277     void push_back(const T& item){
00278         if(size_ == capacity_) {
00279             resize();
00280         }
00281         data_[size_] = item;
00282         ++size_;
00283     }
00284
00285     void append_range(){
00286     }
00287
00288     void pop_back(){
00289         if(size_ > 0){
00290             --size_;
00291         }
00292     }
00293
00294     void resize(){
00295         capacity_ *= 2;
00296         T* new_data = new T[capacity_];
00297         for(size_t i=0; i<size_; i++){
00298             new_data[i] = data_[i];
00299         }
00300         delete[] data_;
00301         data_ = new_data;
00302     }
00303
00304     void resize(size_t capacity){
00305         capacity_ = capacity;
00306         T* new_data = new T[capacity_];
00307         for(size_t i=0; i<size_; i++){
00308             new_data[i] = data_[i];
00309         }
00310         delete[] data_;
00311         data_ = new_data;
00312     }
00313
00314     void swap(Vector& other){
00315         std::swap(size_, other.size_);
00316         std::swap(capacity_, other.capacity_);
00317         std::swap(data_, other.data_);
00318     }
00319 };
00320
00321 //non-member functions
00322 template <typename T>
00323 bool operator==(const Vector<T>& lhs, const Vector<T>& rhs ){
00324     if(lhs.size() != rhs.size()){
00325         return false;
00326     }
00327
00328     for(size_t i = 0; i < lhs.size(); ++i) {
00329         if(lhs.at(i) != rhs.at(i)){
00330             return false;
00331         }
00332     }
00333
00334     return true;
00335 }
00336
00337 template <typename T>
00338 bool operator!=(const Vector<T>& lhs, const Vector<T>& rhs ){
00339     if(lhs.size() != rhs.size()){
00340         return true;
00341     }
00342 }
00343

```

```
00344     for(size_t i = 0; i < lhs.size(); ++i) {  
00345         if(lhs.at(i) != rhs.at(i)){  
00346             return true;  
00347         }  
00348     }  
00349     return false;  
00350 }  
00351 }
```

Index

- ~Vector
 - Vector< T, Allocator >, [8](#)
- allocator_type
 - Vector< T, Allocator >, [6](#)
- append_range
 - Vector< T, Allocator >, [8](#)
- assign
 - Vector< T, Allocator >, [8](#)
- at
 - Vector< T, Allocator >, [8](#), [9](#)
- back
 - Vector< T, Allocator >, [9](#)
- begin
 - Vector< T, Allocator >, [9](#)
- capacity
 - Vector< T, Allocator >, [9](#)
- clear
 - Vector< T, Allocator >, [9](#)
- const_iterator
 - Vector< T, Allocator >, [6](#)
- const_pointer
 - Vector< T, Allocator >, [6](#)
- const_reference
 - Vector< T, Allocator >, [6](#)
- data
 - Vector< T, Allocator >, [9](#), [10](#)
- difference_type
 - Vector< T, Allocator >, [6](#)
- empty
 - Vector< T, Allocator >, [10](#)
- end
 - Vector< T, Allocator >, [10](#)
- erase
 - Vector< T, Allocator >, [10](#)
- front
 - Vector< T, Allocator >, [10](#)
- insert
 - Vector< T, Allocator >, [11](#)
- iterator
 - Vector< T, Allocator >, [7](#)
- max_size
 - Vector< T, Allocator >, [11](#)
- operator!=
 - vector.h, [13](#)
- operator=
 - Vector< T, Allocator >, [11](#)
- operator==
 - vector.h, [13](#)
- operator[]
 - Vector< T, Allocator >, [11](#)
- pointer
 - Vector< T, Allocator >, [7](#)
- pop_back
 - Vector< T, Allocator >, [11](#)
- push_back
 - Vector< T, Allocator >, [11](#)
- reference
 - Vector< T, Allocator >, [7](#)
- reserve
 - Vector< T, Allocator >, [11](#)
- resize
 - Vector< T, Allocator >, [12](#)
- reverse_iterator
 - Vector< T, Allocator >, [7](#)
- shrink_to_fit
 - Vector< T, Allocator >, [12](#)
- size
 - Vector< T, Allocator >, [12](#)
- size_type
 - Vector< T, Allocator >, [7](#)
- swap
 - Vector< T, Allocator >, [12](#)
- value_type
 - Vector< T, Allocator >, [7](#)
- Vector
 - Vector< T, Allocator >, [7](#), [8](#)
- Vector< T, Allocator >, [5](#)
 - ~Vector, [8](#)
 - allocator_type, [6](#)
 - append_range, [8](#)
 - assign, [8](#)
 - at, [8](#), [9](#)
 - back, [9](#)
 - begin, [9](#)
 - capacity, [9](#)
 - clear, [9](#)
 - const_iterator, [6](#)
 - const_pointer, [6](#)
 - const_reference, [6](#)

- data, [9](#), [10](#)
- difference_type, [6](#)
- empty, [10](#)
- end, [10](#)
- erase, [10](#)
- front, [10](#)
- insert, [11](#)
- iterator, [7](#)
- max_size, [11](#)
- operator=, [11](#)
- operator[], [11](#)
- pointer, [7](#)
- pop_back, [11](#)
- push_back, [11](#)
- reference, [7](#)
- reserve, [11](#)
- resize, [12](#)
- reverse_iterator, [7](#)
- shrink_to_fit, [12](#)
- size, [12](#)
- size_type, [7](#)
- swap, [12](#)
- value_type, [7](#)
- Vector, [7](#), [8](#)
- vector.h, [13](#)
 - operator!=, [13](#)
 - operator==, [13](#)